

ModelForge

Compiler Design Phase 1

Name: D.N.S.Vaishnavi

Reg no: 24BCB0093

Abstract

ModelForge is an educational compiler for a small, controlled subset of ONNX feed-forward neural-network models. It will read a model graph, validate supported operators and tensor properties, lower the graph into a custom intermediate representation, apply simple optimizations, and generate readable standalone C++17 inference code.

The generated program will be compared with the original model executed through ONNX Runtime. The purpose is not to reproduce a complete industrial ML compiler. The purpose is to make the stages of model loading, semantic analysis, intermediate representation, optimization, target-code generation, and output verification visible in one manageable project.

1. Problem Statement

Small machine-learning models are usually executed through general-purpose inference runtimes. This is practical, but it hides the translation from a model graph to executable computations and introduces dependencies that are unnecessary for a small demonstration network. A compact compiler is needed to show how a model can be inspected, validated, transformed, and translated into understandable target code.

ModelForge addresses the following question: how can a small ONNX neural-network model be translated into standalone C++ inference code while demonstrating compiler-design concepts such as semantic validation, intermediate representation, optimization, target-code generation, and correctness verification?

2. Motivation

Compiler Design projects often begin with arithmetic expressions or a small programming language. ModelForge applies the same ideas to a machine-learning graph. A graph node consumes tensors, produces tensors, and carries attributes or constants; these relationships must be checked before the computation is translated. This makes the project a practical example of compiler construction applied to a current software format.

The project is deliberately limited to small feed-forward networks. That limitation keeps the implementation understandable for an individual project and makes the final demonstration easier to explain during the viva. Readable generated code and clear diagnostics are more important than broad operator coverage.

3. Objectives

1. Read a supported ONNX neural-network model.
2. Extract graph inputs, outputs, operators, initializers, weights, biases, types, shapes, and relevant attributes.
3. Detect unsupported operators and invalid tensor properties with compiler-style diagnostics.
4. Lower the model into a simplified ModelForge intermediate representation.

5. Implement at least two understandable optimization passes and report their effects.
6. Generate readable standalone C++17 source and header files for inference.
7. Compile and execute the generated program during the integrated project.
8. Compare generated output with the original ONNX model within a documented tolerance.

4. Scope

The first demonstration model is an Iris-style classifier with four input values, a dense layer of eight neurons, a ReLU activation, a dense layer of three neurons, and a three-class Softmax output.

Initial scope: Gemm, MatMul, Add, Relu, Sigmoid, and Softmax

Outside the initial scope: CNN-heavy architectures such as YOLO and ResNet

Initial scope: Static float32 shapes

Outside the initial scope: Transformers, attention, LSTM, and GRU

Initial scope: Small feed-forward classifiers

Outside the initial scope: Dynamic control flow and arbitrary dynamic shapes

Initial scope: C++17 CPU code generation

Outside the initial scope: GPU or CUDA code generation

Initial scope: Textual or simple Graphviz visualization

Outside the initial scope: Complete ONNX operator coverage

Possible future additions include Flatten, Tanh, Identity, an MNIST MLP, memory reuse, SIMD generation, and richer visualization. These will be considered only after the core pipeline is working.

5. Background Study

5.1 ONNX model representation

ONNX represents a model using graph inputs and outputs, operator nodes, initializers, attributes, and tensor types. The graph is serialized using Protocol Buffers and uses versioned operator semantics. This structure makes ONNX a suitable source representation for a teaching compiler: the loader can translate the graph into compiler-owned structures while later stages remain independent of the file format. [1]

5.2 Reference inference

ONNX Runtime provides a standard execution path for ONNX models and has APIs for several languages, including C and C++. ModelForge will use the original model executed by ONNX Runtime as the reference when checking the output of generated C++ code. It is a validation baseline, not the target generated by the compiler. [2]

5.3 Build system

CMake will be used to configure the compiler and the generated target program. Keeping the compiler executable and generated model as separate build targets will make the final demonstration show both stages clearly. [3]

6. Compiler Design Concepts

Model loading: Read the serialized graph and convert it into internal C++ structures.

Semantic analysis: Check supported operators, data types, dimensions, required tensors, and attributes.

Intermediate representation: Represent inputs, constants, dense operations, activations, Softmax, and returns independently of ONNX.

Optimization: Apply constant folding, redundant-operation removal, identity removal, and selected operation fusion.

Target-code generation: Emit readable C++17 functions, buffers, constants, and activation helpers.

Execution: Build the generated source and run standalone inference.

Verification: Compare original and generated outputs using a documented floating-point tolerance.

Error handling: Report unsupported operators, missing tensors, shape mismatches, and invalid attributes.

Lexical analysis and grammar parsing are not central here because ONNX is a serialized graph rather than a text language written by the user. The project instead focuses on the compiler stages that fit graph translation most directly: semantic analysis, IR construction, optimization, and target generation.

7. Proposed Methodology

The compiler will use the following sequence. Each stage passes a defined structure to the next stage so that ONNX-specific logic does not spread into the optimizer or code generator.

1. Load the ONNX model and collect graph metadata, nodes, tensors, initializers, and attributes.
2. Validate operators, data types, dimensions, tensor references, and supported attributes.
3. Lower the validated graph into ModelForge IR instructions and a tensor table.
4. Run optimization passes on the IR and record before-and-after instruction counts.
5. Generate model.cpp and model.h containing constants, buffers, helper functions, and inference logic.
6. Compile and run the generated program, then compare its output with the original ONNX model.

7.1 Validation strategy

The validator will reject operators outside the initial supported set, incompatible matrix dimensions, unsupported data types, missing initializers, duplicate tensor names, missing model outputs, and invalid Softmax axes. A diagnostic will name the failing node or tensor and state the expected condition.

7.2 Optimization strategy

The first passes will be constant folding, redundant-operation elimination, and identity removal. A later pass may fuse a dense operation with a following ReLU when the graph pattern and tensor use are safe. The optimization report will separate an IR simplification from a measured runtime improvement.

7.3 Correctness strategy

The same input will be passed to the original ONNX model and the generated C++ program. Outputs will be compared element by element, and the report will include the maximum difference, class prediction agreement where applicable, the tolerance, and the final pass or fail status.

8. System Architecture

The architecture has a clear source-format boundary. ONNX-specific parsing stops at the loader; the IR, optimizer, and code generator use ModelForge structures. Visualization is optional and reads the graph or IR for explanation only.

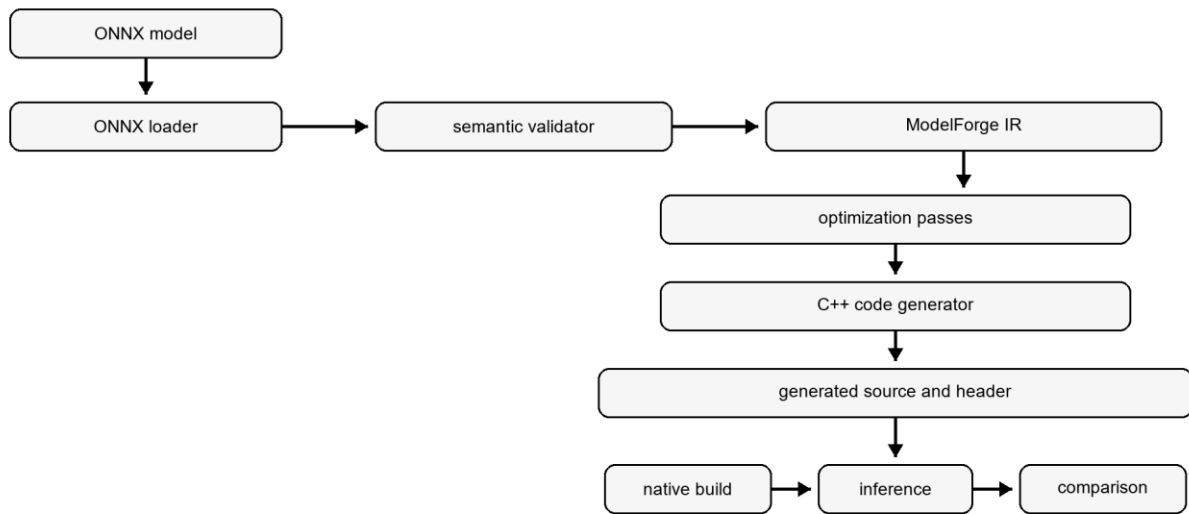


Figure 1. ModelForge compiler flowchart.

ONNX Loader: Read model metadata, graph nodes, tensors, weights, biases, and attributes.

Semantic Validator: Check supported operators, tensor types, dimensions, required parameters, and graph consistency.

ModelForge IR: Provide source-format-independent instructions and tensor metadata.

Optimizer: Apply and count constant folding, redundant-operation removal, identity removal, and optional fusion.

C++ Code Generator: Emit weights, buffers, activation helpers, and the inference function.

Validation Engine: Run both implementations, compare outputs, and report the numerical difference.

Visualization: Optionally display the graph or a before-and-after IR view.

9. Technology Stack

Compiler (C++17): Main implementation language.

Model format (ONNX): Portable source representation for supported models.

Serialization (Protocol Buffers / ONNX protobuf): Read the serialized graph and initializers.

Build (CMake): Configure the compiler and generated target.

Reference inference (ONNX Runtime): Provide baseline output for equivalence checks.

Visualization (Text output or Graphviz): Support explanation of graph and IR structure.

Version control (Git): Record milestones and preserve project history.

10. Conclusion

ModelForge is a feasible and appropriately scoped Compiler Design Laboratory project. It treats an ONNX neural-network graph as a source representation, applies semantic analysis and IR-based transformations, and generates standalone C++ inference code.